

Document number: P1751R0

Date: 2019-06-16

Reply-to: John McFarlane, wg21@john.mcfarlane.name

Audience: SG6, LEWGI

Numeric Type Families

Introduction

This paper introduces the notion of numeric type families which play an important role in facilitating efficient, extensible, generic numeric code. It explains what they are, how they are used and proposes library facilities for accessing and manipulating them.

Definition

The two first paragraphs of [\[basic.fundamental\]](#) in [\[N4810\]](#) introduce the *signed integer types* and the *unsigned integer types* respectively. Together, these constitute the *fundamental integer type family*.

They are supported by optional library aliases (`int32_t` etc.) which provide width guarantees and by type traits (`is_signed` , `make_signed` and `make_unsigned`) which allow transformations between them.

Outside the standard, other numeric type families exist. A recurring pattern is is that their members vary in width and signedness. Some even exhibit a 1-to-1 mapping with the fundamental integer type family. For example, the `SafeInt` class is parameterised on a fundamental integer type:

```
SafeInt<int> a{42};
```

Other families have continuous ranges of widths. For example, `boost::multiprecision::number` has a width parameter:

```
number<cpp_int_backend<32>> b{42};
```

[\[example\]](#)

Applications

Example 1: Generic Arithmetic Functions

There is an increasing demand to make numeric types easier to use in generic code. Consider a library which provides overflow-resistant arithmetic :

```

namespace acme {
    // avoids negative overflow when b>a
    auto minus(uint32_t a, uint32_t b) {
        return int64_t{a}-b;
    }

    // avoids positive overflow when product exceeds range of the operands
    auto multiply(uint32_t a, uint32_t b) {
        return uint64_t{a}*b;
    }
}

```

Now consider all of overloads of the above functions necessary to cover the fundamental integers. Finally, consider the overloads necessary to cover third-party integers such as the `SafeInt` and `Boost.Multiprecision` types. Clearly, function templates are needed.

One facility which would cover the case for fundamental integers is parameterisation of width or digits. We already have `numeric_limits::digits` which is helpful in calculating the range of a number. A complementary setter is frequently requested as a standard library addition.

Boost provides [facilities](#) for choosing a fundamental signed or unsigned type given a number of bits, e.g.:

```

// signed
template<int Bits>
struct int_t
{
    /* Member exact may or may not be defined depending upon Bits */
    typedef implementation-defined-type exact;
    typedef implementation-defined-type least;
    typedef int_fast_t<least>::fast fast;
};

// unsigned
template<int Bits>
struct uint_t
{
    /* Member exact may or may not be defined depending upon Bits */
    typedef implementation-defined-type exact;
    typedef implementation-defined-type least;
    typedef int_fast_t<least>::fast fast;
};

```

This is a step in the right direction and helps out with the `minus` function in our library:

```
namespace acme {  
    // avoids negative overflow when b>a  
    template<class Operand>  
    auto minus(Operand a, Operand b) {  
        using result =  
        boost::int_t<numeric_limits<Operand>::digits+1>::fast;  
        return result{a}-b;  
    }  
  
    // avoids positive overflow when product exceeds range of the operands  
    auto multiply(uint32_t a, uint32_t b) {  
        return uint64_t{a}*b;  
    }  
}
```

But `multiply` is more awkward: we need signed and unsigned overloads to make use of `boost::int_t` and `boost::uint_t`. And it still only works for fundamental integers.

[P0675] proposes a user-customisable metafunction, `set_digits` which addresses both of these concerns with a single type parameter:

```
template<class T, int MinDigits>  
struct set_digits;
```

Common arguments to `T` are `signed` and `unsigned` :

```
set_digits_t<unsigned, 5> a{30}; // an unsigned integer with at least 5  
digits.
```

Two things are worth noting here:

1. The type returned by this metafunction is the same signedness and **the same family** as the input.
2. The terminology is **digits, not bits or width**. This is an important but subtle distinction: width includes the sign bit but digits does not.

Including `set_digits`, [P0675] proposes a full set of user-customizable metafunctions for transforming between types within a family:

- `set_digits` - returns a type with at least the given number of digits
- `digits` - returns the number of digits (equivalent to `numeric_limits::digits`)

- `is_signed` - returns true iff `T` is signed (equivalent to `numeric_limits::is_signed`)
- `add_signedness` - returns a signed equivalent of `T` (like `make_signed` but user-customisable)
- `remove_signedness` - complement to `add_signedness`

Lets see how these helps our `multiply` function :

```
namespace acme {
    // avoids negative overflow when b>a
    template<class Operand>
    auto minus(Operand a, Operand b) {
        constexpr auto operand_digits = digits_v<Operand>;
        using signed_operand = add_signedness_t<Operand>;
        using result = set_digits_t<signed_operand, operand_digits+1>;
        return result{a}-b;
    }

    // avoids positive overflow when product exceeds range of the operands
    template<class Operand>
    auto multiply(Operand a, Operand b) {
        constexpr auto operand_digits = digits_v<Operand>;
        using result = set_digits_t<Operand, operand_digits*2>;
        return result{a}*b;
    }
}
```

Now the functions work with all fundamental integers (except the widest ones, for which there are no valid return types). Better still, we can now provide [a dab of glue](#) and pass Boost.Multiprecision types, among many others.

Example 2: Generic Arithmetic Types

The above example shows safe -- but tedious -- named functions which quickly becomes unreadable. [\[P0828\]](#) proposes the same facility in the form of a numeric type, `elastic_integer`, with a full set of arithmetic operators:

```
template<int MaxDigits, class Narrowest>
class elastic_integer;
```

This makes the same facility far more readable:

```
// elastic_integer<32, signed>{-1}
auto a{elastic_integer{0U}-elastic_integer{1U}};

// elastic_integer<64, unsigned>{UINT_MAX*UINT_MAX}
auto b{elastic_integer{UINT_MAX}*elastic_integer{UINT_MAX}};
```

Example 3: Composite Types

In order to be generally useful, `elastic_integer` also specializes the family metafunctions, e.g.:

```
template<int Digits, class Narrowest, int MinNumBits>
struct set_digits<elastic_integer<Digits, Narrowest>, MinNumBits> {
    using type = elastic_integer<MinNumBits, Narrowest>;
};
```

Now `elastic_integer` itself can start to become a piece in a larger system and we begin to build up a vocabulary for describing components and their interaction. For instance, we can use a 'safe integer' type to wrap -- not only fundamental types as `SafeInt` does but also -- all suitable class-based numeric types.

[P0554] describes an entire system for numeric composition in which the notion of a family will become an essential notion. Introduced in that paper is a more generic alternative to `SafeInt` called `overflow_integer`:

```
template<class Rep, class OverflowTag = contract_overflow_tag>
class overflow_integer;
```

Once `set_digits` and all the other family metafunctions have been specialized for `elastic_integer`, it becomes a family which can be plugged into `overflow_integer`:

```
template<
    int MinDigits,
    class Narrowest = signed,
    class OverflowTag = contract_overflow_tag>
using safe_integer =
    overflow_integer<elastic_integer<MinDigits, Narrowest>, OverflowTag>;
```

Such a **composite numeric type** is hard to digest without some idea of the families involved:

- `Narrowest` is -- by default -- a signed member of the fundamental integer family. The concrete storage for an instantiation of `safe_integer` will be either `int` or something wider than `int`.

Swapping this out for `unsigned` would give equivalent unsigned behavior. Non-fundamental families are accepted but must have the family metafunctions specialized for them.

- `Rep` is the family wrapped by `overflow_integer` and in this case, the choice is `elastic_integer<MinDigits, Narrowest>`. Any change to the digits or signedness of `overflow_integer` will involve an equivalent change in the `elastic_integer` parameters.
- `safe_integer<MinDigits, Narrowest, OverflowTag>` is specialized to become a member of the outermost family. A member of that family might be, `safe_integer<31>` which would give it the same digits and signedness as `int32_t`. (To strain the family metaphor to its limits, if `int` were the matriarch of the fundamental integer family, her default `safe_integer` counterpart would be `safe_integer<31>`.)

Acknowledgements

Special thanks to Davis Herring for extensive and invaluable input on this topic.